Synthetic · Non-proprietary · Deterministic

DRAM Layout Generator

Generates fully synthetic DRAM memory-cell-array GDSII layou
semiconductor inspection dataset generation. No proprietary |
— all geometry is derived from published textbooks and expire

6	10	45	18
GDS LAYERS	VARIATION TYPES	TESTS PASSING	METADATA KEYS

GETTING STARTED

Quick Start

Installation

The generator has three dependencies: `klayout`, `numpy`, and stdlib.

```
# Install into your environment
pip install klayout numpy
# Or with conda
conda activate royl
conda run -n royl python dram_layout_generator.py
```

Simplest usage

```
from dram_layout_generator import generate_dram_layout

# Generates dram.gds + dram_metadata.json with defaults
meta = generate_dram_layout()

print(meta["pixels_per_nm"])          # rasterization sc
print(meta["reference_center_px"])     # ground-truth lo
```

Custom parameters

```
from dram_layout_generator import DRAMParams, DRAMGenerat

p = DRAMParams(
    rows=128, cols=128,
    cell_pitch_nm=80.0,
    cell_pitch_bl_nm=60.0,
    seed=42,
    output_gds="my_array.gds",
    output_json="my_array.json",
)
gen  = DRAMGenerator(p)
meta = gen.generate()
cfg  = gen.get_rasterization_config()   # RasterizationC
```

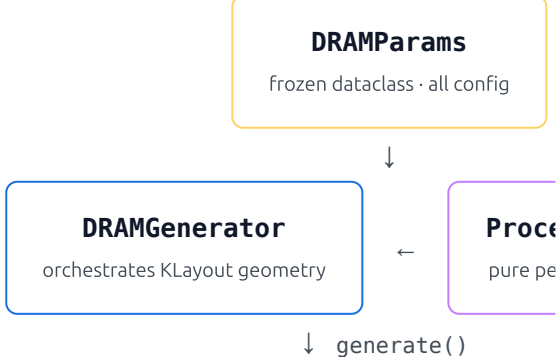
CLI

```
python dram_layout_generator.py \
  --rows 64 --cols 64 \
  --seed 99 \
  --output-gds array.gds \
  --output-json array.json
```

> **Reproducibility:** All randomness flows through a single seeded `np.ra`
> always produces bit-identical GDS and metadata — including reference

DESIGN Architecture

Three cooperating classes in a single file. `ProcessVariation` has zero
dependency — its methods are pure geometry, independently testable



`dram.gds` `dram_metadata.json` `dict (metadat`

Build order inside `generate()`

Shapes are built in dependency order so later steps can reference earlie
(e.g., defects reference WL/BL boxes for CMP dishing):

```
_build_dummy_cells()    # L6: boundary fills
_build_capacitors()     # L1: storage nodes
_build_wordlines()      # L2: horizontal WL stripes
_build_bitlines()       # L3: vertical BL stripes
_build_contacts()       # L4: drain contacts
_add_defects()          # L5: particles, scratches, CMP
                         # → CMP clears L2/L3 and re-inse
# Then: reference patch selection, metadata assembly, fi
```

> **CMP dishing:** `_add_defects` calls `cell.shapes(L_WL).clear()` th
> WL/BL shapes seen after `generate()` differ from those placed during
> tracked in `self._wl_boxes` / `self._bl_boxes` for exactly this reasc

REFERENCE

GDS Layers

Six layers covering the full 1T-1C memory cell stack. Layer numbers mat
KLayout default palette used throughout this document.

LAYER	CONSTANT	PHYSICAL STRUCTURE
`(1,0)`	`LAYER_CAPACITOR`	Storage node capacitor rectangles. E
`(2,0)`	`LAYER_WORDLINE`	Horizontal WL stripes — one per acti dummy border. Gate of access transi
`(3,0)`	`LAYER_BITLINE`	Vertical BL stripes — one per active of access transistor.
`(4,0)`	`LAYER_CONTACT`	Drain contacts — small squares offse of each cell.
`(5,0)`	`LAYER_DEFECT`	Particle contamination, scratches, an markers.
`(6,0)`	`LAYER_DUMMY`	Boundary fills surrounding the activ

periodic boundary conditions for edg

> All coordinates are stored as integer nanometres. `layout.dbu = dbu_` `dbu_nm=1.0` gives 1 DBU = 1 nm.

REFERENCE

DRAMParams Fields

`DRAMParams` is a `frozen=True` dataclass — it cannot be mutated afte construction. All fields have defaults so you can construct with zero arg

Array geometry

FIELD	TYPE	DEFAULT	DESCRI
`rows`	`int`	`64`	Active r
`cols`	`int`	`64`	Active c
`cell_pitch_nm`	`float`	`80.0`	WL-dire
`cell_pitch_bl_nm`	`float`	`60.0`	BL-dire
`wl_width_nm`	`float`	`24.0`	Word-li
`bl_width_nm`	`float`	`18.0`	Bit-line
`contact_size_nm`	`float`	`14.0`	Square
`capacitor_size_nm`	`float`	`30.0`	Square
`dummy_rows`	`int`	`2`	Dummy
`dummy_cols`	`int`	`2`	Dummy

Process variation sigmas

FIELD	TYPE	DEFAULT	DESCF
`overlay_sigma_nm`	`float`	`3.0`	Layer-
`linewidth_sigma_nm`	`float`	`2.0`	CD va
`ler_amplitude_nm`	`float`	`1.5`	Line-e

`ler_period_nm`	`float`	`20.0`	LER sp
`contact_sigma_frac`	`float`	`0.08`	Fracti
`jitter_sigma_nm`	`float`	`1.0`	Cell-p

Defect probabilities

FIELD	TYPE	DEFAULT	DESCRIPT
`p_missing_contact`	`float`	`0.005`	Bernoulli
`p_missing_capacitor`	`float`	`0.003`	Bernoulli
`p_broken_wl`	`float`	`0.02`	Bernoulli
`p_broken_bl`	`float`	`0.02`	Bernoulli
`n_particles`	`int`	`3`	Number c
`n_scratches`	`int`	`2`	Number c
`p_cmp_dishing`	`float`	`0.10`	Fraction c

Output & seed

FIELD	TYPE	DEFAULT	DESCRIF
`output_gds`	`str`	`"dram.gds"`	GDS out
`output_json`	`str`	`"dram_metadata.json"`	JSON si
`seed`	`int`	`42`	RNG see
`dbu_nm`	`float`	`1.0`	DBU = 1

Rasterization fields NEW

These are validated on construction — values must match the benchma exactly.

FIELD	TYPE	DEFAULT	MUST EQUAL
`output_width_px`	`int`	`1000`	1000
`output_height_px`	`int`	`1000`	1000

`reference_width_px`	`int`	`100`	100
`reference_height_px`	`int`	`100`	100
`zoom_ratio`	`int`	`10`	10
`pixels_per_nm`	`Optional[float]`	`None`	—

> **Auto-computation:** when `pixels_per_nm=None`, the scale is set to `m` `array_h_nm)` — the tightest fit that keeps the full active array visible i array (3840 nm × 5120 nm) this yields `≈ 0.1953 px/nm`.

REFERENCE

Process Variations

All variation logic lives in `ProcessVariation`. Methods take `db.Box` ir return perturbed `db.Box` or `List[db.Box]` — no KLayout layout/cell geometry. Applied per the sequence in the build step that calls them.

`overlay_shift(box)`

Shifts a box by Gaussian(0, σ) in both x and y. Simulates layer-to-layer photomask misalignment.

σ ← `overlay_sigma_nm`

`vary_line`

Perturbs lir width. Inwa

σ ← `linew`

`add_ler(box, axis)`

Slices the box into n segments, perturbing each edge independently. Approximates stochastic line-edge roughness from photon-shot and resist blur.

`amp ← ler_amplitude_nm, period ← ler_period_nm`

`vary_cont`

Scales cont centred. M lithography

σ ← `conta`

`jitter_cell(cx, cy)`

Displaces a cell centre by Gaussian(0, σ) in x and y. Models sub-resolution placement error from stage vibration.

σ ← `jitter_sigma_nm`

`is_missi`

Bernoulli d to gate mis

`prob ← p_`

…

`break_line(box, axis)`

Removes a random sub-segment from a WL or BL, returning two surviving pieces. Simulates

`add_parti`

Places n ra the full bou

electromigration or etch failure.

`prob ← p_broken_wl / p_broken_bl`

`n ← n_par`

`add_scratches(bbox, n)`

Places n thin elongated rectangles (horizontal or vertical at random). Simulates CMP-induced surface scratches.

`n ← n_scratches`

`cmp_dishi`

Shrinks line radius r of (from centr geometry -

`fraction ‹`

Variation order for contacts

The spec mandates a specific order that ensures the defect record refle *varied* geometry, not the nominal one:

```
# _build_contacts: correct order
box = vary_contact(box)       # 1. scale CD
box = overlay_shift(box)      # 2. displace
if is_missing(p_missing):    # 3. check missing (uses va
    record_defect(box)
    continue
```

RASTERIZATION EXTENSION

Rasterization Interface

> **New in this version.** The rasterization extension adds benchmark-ready No image rasterization is performed here — the GDS stores physical nm converts using `pixels_per_nm`.

Design principle: resolution independence

The GDS contains only physical nanometre coordinates. The `pixels_pe` factor lives exclusively in metadata. The same GDS file can serve multipl modalities (SEM 1nm/px, optical 10nm/px) by using different scale facto regeneration needed.

RasterizationConfig

Returned by `DRAMGenerator.get_rasterization_config()` after `gen` Contains everything a rasterizer needs with no dependency on `DRAMPa`

```
@dataclass(frozen=True)
class RasterizationConfig:
    gds_file:          str               # path to GDSII
    pixels_per_nm:     float             # rasterization
    origin_nm:         Tuple[float,float] # (x,y) nm at
    width_px:          int               # 1000
    height_px:         int               # 1000
    layers:            List[Tuple[int,int]] # GDS (layer,
    reference_bbox_px: List[int]         # [x0,y0,x1,y1]
    reference_center_px: List[float]    # [cx,cy] ground
```

Reference patch selection

A 100×100 px reference crop window is placed at a uniformly random lo inside the active array, using the seeded RNG (deterministic). The top-le `(rx0, ry0)` is drawn from:

```
ref_w_nm = reference_width_px  / pixels_per_nm
ref_h_nm = reference_height_px / pixels_per_nm

rx0 ~ Uniform(0, array_w_nm − ref_w_nm)
ry0 ~ Uniform(0, array_h_nm − ref_h_nm)
```

> **Ground-truth label:** `reference_center_px` is the exact prediction ta from the seeded generator — no human annotation needed. A model i 100×100 reference crop; its task is to predict `reference_center_px`.

Typical rasterizer integration

```
gen  = DRAMGenerator(params)
meta = gen.generate()
cfg  = gen.get_rasterization_config()

# Your rasterizer uses:
#   cfg.gds_file       — what to render
#   cfg.pixels_per_nm  — rendering scale
#   cfg.origin_nm      — (0, 0) in nm
#   cfg.width_px / height_px  — output size (1000×1000)
#   cfg.layers         — which GDS layers to include

# Ground-truth for the localization benchmark:
gt_cx, gt_cy = cfg.reference_center_px  # pixel-space pre
bbox         = cfg.reference_bbox_px    # [x0, y0, x1, y1
```

RASTERIZATION EXTENSION

Coordinate Utilities

Four pure module-level functions with no KLayout or `DRAMParams` dep
Trivially importable and testable in isolation.

```
from dram_layout_generator import (
    nm_to_pixel, pixel_to_nm,
    bbox_nm_to_pixel, bbox_pixel_to_nm,
)

ppn = 0.25  # pixels per nm

nm_to_pixel(400.0, ppn)                          # → 100.0
pixel_to_nm(100.0, ppn)                          # → 400.0

bbox_nm_to_pixel([10,20,110,220], ppn)         # → [2.5, 5
bbox_pixel_to_nm([2.5,5.0,27.5,55.0], ppn)     # → [10.0,
```

These are the building blocks for any code that needs to translate betw
nm coordinates and image pixel coordinates. The generator uses them i
via private wrappers `self._nm_to_px()` / `self._px_to_nm()`.

REFERENCE

Metadata Schema

`generate()` returns a Python dict and writes an identical JSON sidecar
are listed below.

Original keys (8)

`cell_centers`
List of [x_nm, y_nm] actual centres for every active cell (after jitter). Length =

`wordline_coords`
List of dicts: `{"row": i, "y_nm": y, "x_start_nm": x0, "x_end_nm`

`bitline_coords`
List of dicts: `{"col": j, "x_nm": x, "y_start_nm": y0, "y_end_nm`

bounding_box

Active array extent: `{"x_min_nm": 0, "y_min_nm": 0, "x_max_nm": c rows×pitch}`

defect_locations

List of `{"type": "…", "bbox_nm": [x0,y0,x1,y1]}`. Types: missing_c broken_bl, particle, scratch, cmp_dishing.

params

Full `dataclasses.asdict(DRAMParams)` dump — all generation paramet

gds_file

Path string of the written GDS file.

metadata_file

Path string of the written JSON sidecar.

Rasterization keys (10) NEW

search_image_size_px NEW

`[1000, 1000]` — always fixed.

reference_image_size_px NEW

`[100, 100]` — always fixed.

zoom_ratio NEW

`10` — ratio of search to reference pixel extent. No resampling required whe

pixels_per_nm NEW

Float. Auto-computed as `min(1000/array_w_nm, 1000/array_h_nm)` o

search_bbox_nm NEW

`[0, 0, array_w_nm, array_h_nm]` — active array extent in nm.

search_bbox_px NEW

`[0, 0, 1000, 1000]` — active array extent in pixel coords.

reference_bbox_nm NEW

`[rx0, ry0, rx1, ry1]` — ground-truth reference window in nm, inside t

reference_center_nm NEW

`[cx_nm, cy_nm]` — centre of the reference window in nm.

reference_bbox_px NEW

`[rx0_px, ry0_px, rx1_px, ry1_px]` — ground-truth reference windo

reference_center_px NEW

`[cx_px, cy_px]` — **ground-truth prediction target** for the localization be

DEVELOPMENT

Testing

45 tests in `test_dram_layout.py`, organized by component. Run with:

```
# Full suite
conda run -n royl python -m pytest test_dram_layout.py -v

# Rasterization extension only
conda run -n royl python -m pytest test_dram_layout.py -k

# Single test
conda run -n royl python -m pytest test_dram_layout.py::t
```

TEST GROUPS

GROUP	COUNT	WHAT IT COVERS
DRAMParams	3	Defaults, frozen, custom values
ProcessVariation	10	One test per variation method
Generator init + dummy	3	Layer creation, dummy cell count
Capacitors	2	Count, missing rate
Wordlines	3	Count, coords, broken rate
Bitlines	2	Count, broken rate
Contacts	3	Count, missing, offset from cap
Defects	1	Layer 5 populated
generate()	6	Dict keys, GDS file, JSON file, determin
CMP dishing integration	1	WL geometry actually thinned
Rasterization extension	11	Defaults, validation, auto-scale, explicit patch inside array, center matches bbo

> **Test isolation:** Tests that write files use `tempfile.TemporaryDirectc` `ProcessVariation` pass `sigma=0` / `prob=0` or `prob=1` paramete relying on specific random draws.

Files

FILE	ROLE
`dram_layout_generator.py`	Full implement ProcessVariatic
`test_dram_layout.py`	pytest suite —
`docs/superpowers/specs/2026-08-06-dram-layout-generator-design.md`	Canonical desi extension